

## FASE 1 – OBSERVAR (sin modificar código)

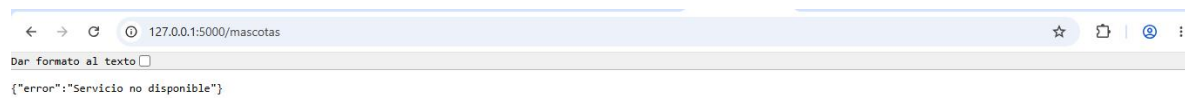
- Apagar el servicio de mascotas
- Hacer varias peticiones al gateway
- Revisar logs

### Responder:

El sistema me muestra 2 fallos a mascotas



Error del sistema que envía el sistema al comienzo



circuito de usuario bloqueado



El sistema me muestra 3 fallos a mascotas

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACION  TERMINAL  PUERTOS

PS C:\pet_shop_clase_sd_g_mayo> docker logs -f pet_shop_clase_sd_gmayo-gateway-1
Fallo mascotas 1
172.19.0.1 - - [10/May/2026 02:44:14] "GET /mascotas HTTP/1.1" 503 -
Fallo mascotas 2
172.19.0.1 - - [10/May/2026 02:44:20] "GET /mascotas HTTP/1.1" 503 -
172.19.0.1 - - [10/May/2026 02:44:42] "GET /mascotas HTTP/1.1" 200 -
Fallo mascotas 1
172.19.0.1 - - [10/May/2026 02:44:57] "GET /mascotas HTTP/1.1" 503 -
Fallo mascotas 2
172.19.0.1 - - [10/May/2026 02:45:03] "GET /mascotas HTTP/1.1" 503 -
Fallo mascotas 3
Circuito abierto
172.19.0.1 - - [10/May/2026 02:45:05] "GET /mascotas HTTP/1.1" 503 -
Fallo mascotas 4
Circuito abierto
172.19.0.1 - - [10/May/2026 02:45:06] "GET /mascotas HTTP/1.1" 503 -
[]
```

Sistema corriendo

```
<  →  ↻  localhost:5000/usuarios  📄 ☆ 📄 | 🔍 ⋮

Dar formato al texto ☐

[{"id":1,"nombre":"Yordin"}, {"id":2,"nombre":"Josés"}]
```

Error del sistema que envía el sistema al comienzo

```
<  →  ↻  localhost:5000/usuarios  📄 ☆ 📄 | 🔍 ⋮

Dar formato al texto ☐

{"error":"Servicio no disponible"}
```

## circuito de usuario bloqueado



El sistema me muestra 3 fallos a usuarios y el nombre del circuito si es abierto o cerrado

```
PS C:\pet_shop_clase_sd_g mayo> docker ps
PS C:\pet_shop_clase_sd_g mayo> docker logs -f pet_shop_clase_sd_gmayo-gateway-1
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.19.0.5:5000
Press CTRL+C to quit
172.19.0.1 - - [10/May/2026 02:33:20] "GET /usuarios HTTP/1.1" 200 -
172.19.0.1 - - [10/May/2026 02:33:21] "GET /usuarios HTTP/1.1" 200 -
Fallo usuarios 1
172.19.0.1 - - [10/May/2026 02:33:39] "GET /usuarios HTTP/1.1" 503 -
Fallo usuarios 2
172.19.0.1 - - [10/May/2026 02:33:45] "GET /usuarios HTTP/1.1" 503 -
Fallo usuarios 3
Circuito abierto
172.19.0.1 - - [10/May/2026 02:33:51] "GET /usuarios HTTP/1.1" 503 -
172.19.0.1 - - [10/May/2026 02:33:55] "GET /usuarios HTTP/1.1" 503 -
172.19.0.1 - - [10/May/2026 02:33:59] "GET /usuarios HTTP/1.1" 503 -
```

Comienzo del sistema resumen los dos servicios arriba

```
localhost:5000/resumen

{
  "mascotas": {
    "mascotas": [
      [1, "Firulaís",
        "perro",
        3]
    ]
  },
  "usuarios": [
    {
      "id": 1,
      "nombre": "Yordin"
    },
    {
      "id": 2,
      "nombre": "Joses"
    }
  ]
}
```

Si bajo un servicio me muestra el error

```
localhost:5000/resumen

{
  "mascotas": {
    "mascotas": [
      [1, "Firulaís",
        "perro",
        3]
    ]
  },
  "usuarios": [
    {
      "error": "no disponible"
    },
    503
  ]
}
```

### ¿Qué hace el sistema actualmente?

El sistema implementa un **API Gateway** con un patrón de **Circuit Breaker** que actúa como punto de entrada único para los microservicios. Actualmente:

1. **Enrutamiento de peticiones:** El gateway redirige las solicitudes a los microservicios correspondientes:
  - GET /usuarios → Microservicio de usuarios
  - GET /mascotas → Microservicio de backend (mascotas)
2. **Monitoreo de fallos:** El sistema lleva un conteo de los errores en las comunicaciones entre servicios.
3. **Registro de eventos:** Cada fallo queda registrado en los logs del gateway para facilitar la depuración.

### ¿Se protege o insiste?

El sistema **hace ambas cosas de forma inteligente**: primero insiste de manera limitada y luego se protege.

| Estado                          | Comportamiento                                    |
|---------------------------------|---------------------------------------------------|
| <b>Primer fallo</b>             | Insiste, cuenta el error y reintenta              |
| <b>Segundo fallo</b>            | Insiste nuevamente, cuenta el error               |
| <b>Tercer fallo</b>             | Último intento, luego activa la protección        |
| <b>Cuarto fallo en adelante</b> | No insiste, responde inmediatamente con error 503 |

## Fase 2

### Respuesta para tu trabajo

#### ¿Cada servicio tiene su propio contador de fallos?

Sí, se implementó un diccionario estados donde cada servicio (usuarios, mascotas) tiene su propio contador independiente.

#### ¿El circuito se abre de forma independiente por servicio?

Sí, se utiliza la variable abierto dentro de cada estado. El circuito de usuarios puede estar abierto mientras el de mascotas permanece cerrado.

#### ¿Qué pasa si falla un servicio pero el otro sigue funcionando?

El sistema sigue operando normalmente para el servicio saludable. Por ejemplo, si usuarios está caído, /usuarios devuelve 503 pero /mascotas sigue funcionando sin problemas.

## FASE 3 – INVESTIGAR (Half-Open)

### ¿Qué significa "half-open"?

El estado **Half-Open** es una fase de transición y prueba donde el circuito breaker permite el paso de **un número limitado de peticiones** para verificar si el servicio que había fallado ya se ha recuperado .

Es como "asomar la cabeza" para comprobar si el peligro ya pasó:

- **Cerrado (Closed)** → Todo funciona, pasan todas las peticiones
- **Abierto (Open)** → Todo falla, se bloquean todas las peticiones
- **Half-Open** → "Modo prueba": pasan solo algunas peticiones para testear

### ¿Cuándo se vuelve a intentar una llamada?

Después de que el circuito permanece en estado **Open** durante un tiempo determinado (llamado *reset timeout* o *recovery timeout*), automáticamente pasa a estado **Half-Open** y comienza a permitir peticiones de prueba .

### Configuración típica:

- recovery\_timeout: 30 segundos en Open
- Al cumplirse el timeout → cambia a Half-Open
- En Half-Open se permiten N peticiones de prueba (ej: 1, 3 o 5)

#### ¿Qué pasa si el servicio vuelve a fallar?

Si durante el estado **Half-Open** alguna de las peticiones de prueba falla, el circuito **reacciona inmediatamente**:

1. **Vuelve a abrirse (Open)** - El circuito transiciona nuevamente a estado Open
2. **Reinicia el contador** - El timeout de recuperación empieza de nuevo
3. **Sigue bloqueando peticiones** - Hasta que pase otro timeout y se reintente

Debes apagar el backend:

```
docker stop pet_shop_clase_sd_gmayo-backend-1
```

Debes apagar usuarios:

```
docker stop pet_shop_clase_sd_gmayo-usuarios-1
```

### 3. Probar /resumen

/resumen depende de:

[Juntar a ChatGPT](#)

Entonces puedes apagar cualquiera de los dos:

opción A

PowerShell

```
docker stop pet_shop_clase_sd_gmayo-backend-1
```

○

opción B

PowerShell

```
docker stop pet_shop_clase_sd_gmayo-usuarios-1
```